An Experimental Comparison of Sorting Algorithms: Bridging Theory and Real-World Performance

Omar Lekhal
Year 1, Group 2
Department of Computer Science
West University of Timişoara
omar.lekhalXX@e-uvt.ro

April 2026

Abstract

Sorting is one of the most thoroughly studied problems in computer science, yet the gap between textbook complexity analysis and observed wall-clock performance remains a source of practical confusion. This paper presents a systematic empirical evaluation of nine sorting algorithms—Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, Quick Sort, Heap Sort, Counting Sort, a Linked-List Merge Sort, and Python's built-in TimSort—across seven input distributions and seven input sizes ranging from 50 to 10 000 000 elements. The central question is whether the theoretical divide between $O(n^2)$ and $O(n \log n)$ algorithms manifests as a clear, observable boundary under real hardware constraints. Our results confirm this divide emphatically: all three $O(n^2)$ algorithms exceed a 10-second hard timeout at $n = 10\,000$, while every $O(n \log n)$ algorithm handles the same size in under 20 ms. Beyond this headline finding, the experiments reveal important second-order effects: TimSort exploits nearly-sorted structure to achieve near-linear speed (sorting 10 000 000 already-sorted integers in 145 ms), Counting Sort dominates on large integer inputs (74 ms at $n = 10^6$) but is inapplicable to floating-point or string data, and pure Python implementations of Merge Sort and Heap Sort are outperformed by TimSort by a factor of 10 to 20 at the same asymptotic complexity. The paper provides a reproducible benchmark harness, discusses the role of constant factors and data-awareness in practical algorithm selection, and offers guidelines for choosing the appropriate algorithm given real-world constraints.

Contents

1 Introduction **3**

2 Formal Description of the Problem and Algorithms **4**
2.1 The Sorting Problem 4
2.2 Complexity Summary 4

2.3 Key Algorithmic Properties 5

3 Implementation and Experimental Setup **5**
3.1 Implementation Details 5
3.2 Benchmark Harness and Timeout Mechanism 6
3.3 Input Distributions and Sizes 6

4 Experimental Results and Analysis **7**
4.1 The $O(n^2)$ Performance Wall 7
4.2 Performance of $O(n \log n)$ Algorithms 7
4.3 Counting Sort and Data-Type Constraints 8
4.4 TimSort on Adaptive Profiles 8

5 Related Work **9**

6 Conclusions and Future Work **9**

1 Introduction

Motivation

Sorting is arguably the single most important subroutine in computing. From database query planners and file-system indexers to machine-learning pipelines and network routing tables, nearly every large-scale system relies on efficient ordering at its core. Textbooks such as [CLRS09] classify sorting algorithms primarily by their asymptotic complexity: the quadratic $O(n^2)$ class (Bubble, Selection, Insertion) versus the quasi-linear $O(n \log n)$ class (Merge, Quick, Heap, TimSort). This classification is correct and essential—yet it abstracts away constant factors, memory access patterns, branch prediction, and the statistical properties of real input data, all of which substantially influence the time a program actually takes to run.

The question this paper addresses is therefore a practical one: *how does theoretical complexity translate to real execution time on a modern CPU, across a variety of input types and sizes?* This is not merely an academic curiosity. A developer choosing between algorithms for a production system must understand not just big-O bounds, but at what input size those bounds become the determining factor, how adaptive algorithms behave on partially ordered data, and which algorithms simply cannot handle the data type at hand.

Informal Description of the Experiment

To answer the question above, we implemented nine sorting algorithms in Python 3 and benchmarked them across a grid of configurations. The input sizes range from $n = 50$ (where even quadratic algorithms are imperceptibly fast) to $n = 10\,000\,000$ (where constant factors and memory bandwidth become decisive). Seven input distributions were tested: uniformly random integers, already-sorted integers, reverse-sorted integers, almost-sorted integers, integer arrays with low cardinality ("flat"), uniformly random floating-point numbers, and uniformly random strings. This breadth is intended to stress-test the data-awareness of each algorithm.

A key design decision is the handling of slow algorithms at large sizes. Rather than allowing a single run to consume hours, we imposed a *10-second hard timeout* per algorithm-size-profile combination: any run still executing after 10 seconds is killed and recorded as `TIMEOUT`. This timeout has a clear physical interpretation—an algorithm that cannot sort 10 000 random integers within 10 seconds is not usable in a latency-sensitive real-time system. Conversely, at small sizes ($n \leq 1000$), we repeated each measurement thousands of times and reported the mean elapsed time, to suppress noise and ensure statistical relevance.

A Running Example

As a concrete illustration throughout this paper, we will follow the fate of a list of $n = 10$ integers $[7, 3, 9, 1, 5, 8, 2, 6, 4, 0]$. Bubble Sort will scan this list up to nine times, performing up to 45 comparisons. Merge Sort will divide it into singleton lists and merge them in $\lceil \log_2 10 \rceil = 4$ levels, performing at most 24 comparisons. At this tiny scale, both approaches finish in microseconds. As we scale to millions of elements, the difference between $O(n^2)$ and $O(n \log n)$ becomes the difference between unusable and fast.

Contributions

The contributions of this paper are:

1. A publicly reproducible benchmark harness in Python with a multiprocessing-based timeout mechanism.
2. Empirical evidence confirming and quantifying the $O(n^2)$ performance wall at $n = 10\,000$ under a 10-second time budget.
3. A cross-distribution analysis revealing how Bubble Sort becomes *linear* on sorted input while Selection Sort remains quadratic regardless of order.
4. A comparison of TimSort against Merge Sort and Heap Sort that quantifies the practical advantage of adaptive, C-implemented sorting over pure Python equivalents at the same asymptotic class.

Paper Structure

The remainder of the paper is organized as follows. Section 2 defines the sorting problem formally and summarises the complexity of each algorithm. Section 3 describes the implementation, benchmark harness, and experimental setup. Section 4 presents and analyses the experimental results, including the data table and cross-profile comparisons. Section 5 situates our findings in the context of prior work. Section 6 summarises the findings and outlines future directions.

2 Formal Description of the Problem and Algorithms

2.1 The Sorting Problem

Definition 1 (Sorting Problem). *Let $A = \langle a_1, a_2, \ldots, a_n \rangle$ be a sequence of n elements drawn from a totally ordered set $(U, \leq)$. A sorting algorithm produces a permutation $A' = \langle a_{\pi(1)}, a_{\pi(2)}, \ldots, a_{\pi(n)} \rangle$ of A such that $a_{\pi(1)} \leq a_{\pi(2)} \leq \cdots \leq a_{\pi(n)}$.*

This formulation, standard in the literature [CLRS09], admits any totally ordered universe U: integers, floating-point numbers, strings ordered lexicographically, etc. The requirement that U be totally ordered is significant: it excludes Counting Sort from being applicable to arbitrary types, as will be seen in Section 4.

2.2 Complexity Summary

Table 1 summarises the time and space complexity of the nine algorithms studied. All complexities use n to denote the number of elements. The notation k in Counting Sort refers to the range of values: $k = \max(A) - \min(A)$.

Table 1: Theoretical complexity of the nine sorting algorithms. Best, average, and worst-case time complexities are given in O-notation. Space complexity refers to auxiliary memory beyond the input array.

Algorithm	**Best**	**Average**	**Worst**	**Space**
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$
Counting Sort	$O(n + k)$	$O(n + k)$	$O(n + k)$	$O(k)$
Linked-List Merge	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
TimSort	$O(n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$

2.3 Key Algorithmic Properties

The $O(n^2)$ algorithms share a structural feature: their inner loop performs work proportional to the *current unsortedness* of the input. For Bubble Sort and Insertion Sort, this means they degrade gracefully on nearly-sorted data (best case $O(n)$), while Selection Sort always performs $\frac{n(n-1)}{2}$ comparisons regardless of input order [CLRS09].

Among the $O(n \log n)$ algorithms, Merge Sort [CLRS09] achieves guaranteed worst-case performance at the cost of $O(n)$ auxiliary memory. Quick Sort, as described in [CLRS09], achieves excellent average-case performance with $O(\log n)$ stack space; our implementation uses a median-of-three pivot strategy to reduce the probability of worst-case $O(n^2)$ behaviour. Heap Sort is in-place and asymptotically optimal but has poor cache performance due to non-sequential memory access during sift-down operations.

TimSort [Peters02] is a hybrid algorithm combining Merge Sort with Insertion Sort. It identifies already-sorted *runs* in the input and merges them. On fully sorted input, TimSort operates in $O(n)$ time by detecting a single run spanning the entire array; this property is directly observable in our experimental results.

Counting Sort operates in $O(n + k)$ time, where k is the value range. When $k = O(n)$, this is linear and faster than any comparison-based algorithm. However, Counting Sort requires integer keys and a bounded range; it is fundamentally inapplicable to floating-point or string data.

3 Implementation and Experimental Setup

3.1 Implementation Details

All nine algorithms were implemented in Python 3.11, running on an Apple M1 system with 16 GB of RAM under macOS. Using a single language for all algorithms ensures that the measurements reflect algorithmic differences rather than inter-language implementation effects. Each algorithm operates on a fresh copy of the input array, so no in-place mutation carries over between measurements.

Two implementation details deserve explicit mention. First, Python's default recursion limit of 1000 frames would cause a `RecursionError` when sorting large arrays with Merge Sort or Quick Sort. The benchmark therefore sets `sys.setrecursionlimit(300_000)`, providing sufficient stack depth for all tested sizes. Second, a global `SEED = 42` is passed to Python's `random` module at startup, ensuring that the exact same input arrays are generated in every run of the benchmark. This makes the results fully reproducible.

The Linked-List Merge Sort is a structurally distinct implementation of the same Merge Sort algorithm: the input array is first converted into a singly-linked list, sorted via recursive splitting and merging of list nodes, and then converted back to an array. This variant is included to measure the overhead of pointer-chasing versus contiguous-array access, isolating the effect of data structure choice on an otherwise identical algorithm.

3.2 Benchmark Harness and Timeout Mechanism

A central challenge in benchmarking algorithms with vastly different performance profiles is that a naive sequential loop would stall on the first slow algorithm encountered at large sizes. We address this with a *hard-timeout harness* based on Python's `multiprocessing` module:

```python
def benchmark_with_timeout(func, gen, n, runs, timeout=10):
    manager = mp.Manager()
    return_dict = manager.dict()
    p = mp.Process(target=_worker_process,
                   args=(func, gen, n, runs, return_dict))
    p.start()
    p.join(timeout)
    if p.is_alive():
        p.terminate()
        p.join()
        return "TIMEOUT"
    return return_dict.get('result', "ERR")
```

Listing 1: Timeout harness using multiprocessing.

Each combination of (algorithm, data profile, size) runs in a separate operating-system process. If that process has not completed within 10 seconds, it is hard-killed via `p.terminate()` and the result is recorded as `TIMEOUT`. This approach has two important properties: (i) a timed-out run does not block subsequent measurements, and (ii) the timeout is enforced at the OS level, making it immune to Python-level delays (e.g. GIL contention, garbage collection).

3.3 Input Distributions and Sizes

Seven input distributions were designed to stress-test different aspects of each algorithm's behaviour:

Random Integers drawn uniformly at random from $[0, 100\,000]$. This is the canonical "average case" for comparison-based sorts.

Sorted The sequence $0, 1, 2, \ldots, n-1$. Reveals best-case behaviour for adaptive algorithms.

Reverse Sorted The sequence $n, n-1, \ldots, 1$. Worst case for Bubble and Insertion Sort.

Almost Sorted A sorted sequence with 2% of elements randomly swapped. Intermediate case for adaptive algorithms.

Flat Integers drawn uniformly from $[0, 5]$, producing arrays with high repetition. Tests behaviour with many duplicate keys.

Random Float Floating-point values in $[0, 10^5)$. Excludes integer-only algorithms such as Counting Sort.

Random String Six-character lowercase ASCII strings. Tests lexicographic comparison and further excludes Counting Sort.

The seven sizes tested are $n \in \{50, 100, 1000, 10\,000, 100\,000, 1\,000\,000, 10\,000\,000\}$. For small sizes ($n \leq 1000$), the benchmark runs each algorithm 100 to 20 000 times and reports the mean, to average out system noise. For large sizes ($n \geq 100\,000$), a single timed run suffices—the execution time is large enough that individual measurement variability is negligible.

4 Experimental Results and Analysis

4.1 The $O(n^2)$ Performance Wall

The most striking result of the experiment is the sharp performance cliff exhibited by all three $O(n^2)$ algorithms at $n = 10\,000$. On the Random integer profile, Bubble Sort completes in 275 μs at $n = 100$ and 29.97 ms at $n = 1000$—a 100× growth for a 10× increase in size, exactly matching the quadratic prediction. At $n = 10\,000$, Bubble Sort exceeds the 10-second timeout without completing. Insertion Sort and Selection Sort exhibit the same pattern: both finish at $n = 1000$ (12.02 ms and 12.73 ms, respectively) and both time out at $n = 10\,000$.

This result has a concrete practical interpretation: no $O(n^2)$ Python implementation in this study is usable for sorting more than a few thousand elements in a real-time context. Even at $n = 1000$, the 12–30 ms range is already borderline for latency-sensitive applications. A developer relying on Bubble Sort for a dataset that occasionally grows to 10 000 items would observe sporadic 10-second stalls—a severe user-visible regression.

There is, however, a notable exception within the $O(n^2)$ family: the influence of input order. On *already-sorted* data, Bubble Sort degrades to $O(n)$ due to its early-exit optimisation. Our results confirm this: on the Sorted profile, Bubble Sort completes even at $n = 10\,000\,000$ in only 407 ms. Insertion Sort behaves similarly: 624 ms at $n = 10\,000\,000$ on sorted input. By contrast, Selection Sort—which has no such adaptive mechanism—times out at $n = 10\,000$ even on sorted data, confirming its unconditional $\Theta(n^2)$ complexity.

4.2 Performance of $O(n \log n)$ Algorithms

Table 2 presents the measured execution times for three representative algorithms on the Random integer distribution at the three most diagnostically informative sizes.

Table 2: Execution time on the *Random integer* profile for three algorithms at three input sizes. `TIMEOUT` indicates the run exceeded the 10-second limit without completing.

Algorithm	$n = 100$	$n = 10\,000$	$n = 1\,000\,000$
Bubble Sort	$275.67\,\mu s$	TIMEOUT	TIMEOUT
Merge Sort	$84.00\,\mu s$	$13.90\,ms$	$2.034\,s$
TimSort	$4.70\,\mu s$	$850.13\,\mu s$	$143.58\,ms$

At $n = 100$, Merge Sort is already faster than Bubble Sort by a factor of roughly 3.3×, even though both complete in sub-millisecond time. By $n = 10\,000$, Merge Sort finishes in 13.90 ms—comfortably within any reasonable time budget—while Bubble Sort does not finish at all. At $n = 1\,000\,000$, Merge Sort requires 2.034 seconds. TimSort, at the same complexity class, completes the same task in 143.58 ms—a 14× improvement. This disparity reflects not a difference in asymptotic complexity but in constant factors: TimSort is implemented in C within CPython and exploits adaptive run-detection, while our Merge Sort is pure Python with per-element function call overhead.

Among the $O(n \log n)$ algorithms at $n = 10^6$ on Random data, the observed ranking is: TimSort (143 ms) < Quick Sort (1.49 s) < Merge Sort (2.03 s) < Linked-List Merge Sort (2.17 s) < Heap Sort (3.09 s). Heap Sort's poor showing despite its optimal worst-case complexity is consistent with known cache-unfriendliness: sift-down operations jump between distant memory locations, stressing the hardware prefetcher.

4.3 Counting Sort and Data-Type Constraints

Counting Sort stands apart from all other algorithms in both its strengths and its limitations. On the Random integer profile at $n = 10^6$, it completes in 74.21 ms—faster than every other algorithm including TimSort. At $n = 10\,000\,000$, it completes in 618.67 ms, while all other general-purpose algorithms either time out or take over a second. This performance advantage is a direct consequence of its $O(n + k)$ complexity: for random integers in $[0, 100\,000]$, $k = 100\,000 = O(n)$ when n is large, yielding effectively linear behaviour.

However, on the Random Float and Random String profiles, Counting Sort is recorded as `UNSUPPORTED` at every size. This is not a performance failure but a semantic one: Counting Sort requires keys to be non-negative integers with a bounded range, and the algorithm implementation raises a `TypeError` when applied to floating-point or string values. This makes Counting Sort a *domain-specific* optimisation rather than a general-purpose tool.

4.4 TimSort on Adaptive Profiles

The most dramatic single result in the experiment is TimSort on the Sorted profile. At $n = 10\,000\,000$ (10 million already-sorted integers), TimSort completes in 145.01 ms. For context, Quick Sort on the same profile and size times out. This result directly demonstrates the adaptive nature of TimSort: by detecting that the entire input is a single ascending run, it reduces the sorting problem to a single $O(n)$ scan, rather than performing $O(n \log n)$ comparisons and merges.

On the Almost Sorted profile, TimSort similarly outperforms all competitors, completing $n = 10^7$ in under 200 ms, while Merge Sort times out. This confirms that TimSort's advantage on ordered data is not limited to perfectly sorted inputs but extends to any input with significant local ordering—a common property in real-world datasets such as log files, database tables with recent insertions, or already partially processed data streams.

5 Related Work

The asymptotic complexity results cited throughout this paper—particularly for Merge Sort, Quick Sort, and Heap Sort—originate in the standard algorithmic reference by Cormen et al. [CLRS09], which provides formal proofs of correctness and complexity for all three algorithms. The theoretical lower bound of $\Omega(n \log n)$ for comparison-based sorting, also established in [CLRS09], frames the context within which all comparison-based algorithms in this study operate.

TimSort was designed by Tim Peters [Peters02] and subsequently adopted as the standard sorting algorithm in Python and Java. Its adaptive behaviour on nearly-sorted data was analysed formally by Auger et al. [ANV15], who showed that TimSort achieves $O(n \log r)$ comparisons where r is the number of runs in the input—a refinement of the standard $O(n \log n)$ bound.

The question of sorting performance under hardware constraints has been addressed extensively in the context of parallel and GPU-based sorting. Satish et al. [Sat09] demonstrated radix-sort-based approaches achieving throughputs an order of magnitude beyond what CPU-based algorithms can sustain, by exploiting the massive parallelism of GPU architectures. While our work targets single-threaded CPU performance, the contrast with GPU-based results underscores the importance of hardware-specific algorithm design. Similarly, Hirschberg [Hir78] established foundational results on parallel sorting that motivate the study of per-element work reduction—a theme also present in TimSort's run-detection.

Our study complements this prior work by providing a single-platform, cross-algorithm, cross-distribution benchmark that is directly reproducible by students and practitioners without specialised hardware.

6 Conclusions and Future Work

This paper set out to determine whether and how theoretical asymptotic complexity bounds manifest as observable performance differences on a modern CPU. The experimental results provide a clear and unambiguous answer.

The $O(n^2)$ wall is real, sharp, and practically consequential. All three quadratic algorithms—Bubble, Selection, and Insertion Sort—exceed a 10-second time budget at $n = 10\,000$ on random integer input. For datasets of this size or larger, no quadratic algorithm should be used unless the input is guaranteed to be nearly sorted (in which case Insertion Sort can be competitive) or the algorithm is serving as a base case within a hybrid approach.

Among the $O(n \log n)$ algorithms, the practical recommendation is clear: TimSort, Python's built-in `sorted()`, should be the default choice. It is 10–20× faster than pure-Python Merge Sort or Heap Sort at the same asymptotic complexity, and it exploits sorted structure to achieve near-linear performance on real-world datasets that are often partially ordered. The only scenario where an alternative is warranted is Counting Sort for large integer datasets with a bounded range: there it achieves true linear time and outperforms TimSort by a factor of roughly 2× at $n = 10^6$.

The following guidelines summarise the practical conclusions of this study:

$n < 100$: Any algorithm is acceptable. Insertion Sort's simplicity and low overhead make it a reasonable choice for tiny arrays.

$100 \leq n < 10\,000$: Use an $O(n \log n)$ algorithm. TimSort is the default unless implementation constraints require a custom sort.

$n \geq 10\,000$: $O(n^2)$ algorithms are unusable in real-time contexts. Use TimSort (general purpose) or Counting Sort (bounded integers only).

Partially ordered input: TimSort is especially well-suited; it is the only tested algorithm that completes $n = 10^7$ sorted integers in under 200 ms.

String or float data: Counting Sort is inapplicable. TimSort is the recommended default.

Several directions for future work present themselves. First, the benchmark should be extended to multi-threaded and distributed settings; the work of Satish et al. [Sat09] suggests that GPU-based radix sort would dominate all algorithms tested here at large scales. Second, the effect of the Python GIL on multi-process benchmarking could be studied in more depth. Third, the Linked-List Merge Sort overhead could be quantified in terms of cache miss rates using hardware performance counters, providing a mechanistic explanation for the 7% overhead observed relative to array-based Merge Sort.

References

[CLRS09] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein. *Introduction to Algorithms*, 3rd edition. MIT Press, 2009.

[Peters02] T. Peters. *TimSort — Description and Python implementation.* Python mailing list, 2002. `https://bugs.python.org/issue5371`

[ANV15] N. Auger, C. Nicaud, and C. Pivoteau. Merge complexity of sorting algorithms. In *Proceedings of the 42nd International Colloquium on Automata, Languages, and Programming (ICALP)*, 2015.

[Sat09] N. Satish, M. Harris, and M. Garland. Designing efficient sorting algorithms for manycore GPUs. In *2009 IEEE International Symposium on Parallel & Distributed Processing*, pages 1–10. IEEE, 2009.

[Hir78] D. S. Hirschberg. Fast parallel sorting algorithms. *Communications of the ACM*, 21(8):657–661, 1978.